Two-Variable CD vs One-Variable DCD

Project Work in Optimization Techniques for Machine Learning

Riccardo Fantechi

April 14, 2026

Abstract

The main object of this project is to implement two decomposition algorithms for training Linear Support Vector Machines via Dual Coordinate Descent (DCD). Specifically the one-variable Dual Coordinate Descent (DCD) method proposed by Hsieh et al. [1] and the two-variable coordinate descent (2-CD) method proposed by Chiu, Lin and Lin [2] both applied to the unbiased L2-loss SVM dual formulation. All code can be found in the Github repository[1].

1 Introduction

Support Vector Machines (SVM) are among the most well-established methods for binary classification. Given a set of labeled instances, an SVM seeks a linear decision boundary that maximize the margin between the two classes. SVMs can be combined with kernel function to handle non-linear problems effectively. However, for large-scale application involving thousands or millions of instances, kernel SVMs become computationally intractable: the kernel matrix alone requires $O(l^2)$ memory, where l is the number of training samples.

In many practical scenarios, data is high-dimensional but linear models perform comparably to kernel ones. In these cases, linear SVMs offer an alternative: they avoid entirely the kernel matrix and scale well with both the number of instances and features.

The standard approach is to solve the dual formulation of the SVM problem. The dual involves l variables $\alpha_i \geq 0$, one per training instance, and its objective function is twice differentiable. A key insight for linear SVMs is that the gradient of the dual objective can be evaluated in $O(\bar{n})$ operations where $\bar{n}$ is the average number of non-zero features per instance by maintaining the weight vector

$$w = \sum_i \alpha_i y_i x_i$$

throughout the optimization.

This is what makes coordinate descent (CD) methods particularly attractive: rather than solving the full problem at once, they update one or a few dual variables at a time, with each update being cheap and producing a closed form solution.

The one-variable CD method for linear SVM formalized by Hsieh et al. (2008) [1], who showed that cycling through all variables with random permutations achieves linear convergence. More recently Chiu, Lin and Lin (2020) [2] extended this framework to a two-variable CD method, updating pairs of variables simultaneously. The two-variable approach requires solving a small two-dimensional quadratic subproblem at each step, which is more involved but leads to faster convergence in terms of the number of CD steps, particularly for difficult problem instances with large penalty parameter C.

The specific objective of this project are:

1. Implement the one-variable DCD algorithm of Hsieh[1]
2. Implement the two-variable CD method of Chiu et al.[2]
3. Verify correctness by comparing test accuracy against scikit-learn's `LinearSVC`
4. Compare the two methods by plotting the relative objective gap against runtime and number of CD steps.

This report is organized as follows. Section 2 presents the SVM primal and dual formulation. Section 3 describes the one-variable DCD algorithm. Section 4 develops the two-variable CD method, with detailed treatment of the subproblem solution. Section 5 discuss the implementation details. Section 6 presents experimental results. Section 7 concludes.

2 Problem Formulation

2.1 Primal Problem

Given a training set of l instance-label pairs (x_i, y_i), where $x_i \in \mathbb{R}^n$ and $y_i \in \{-1, +1\}$, a linear SVM (soft margin formulation) solves the following risk minimization problem:

$$\min_{w,b} \frac{1}{2} w^T w + C \sum_{i=1}^{l} \xi(w, b; x_i, y_i)$$

[1] `https://github.com/Riccardo1451/Two-Variable-Dual-Coordinate-for-Linear-SVM/`

where $C > 0$ is a penalty parameter that controls the trade off between margin maximization and training error, and ξ is a loss function. There are two common choices:

- $\xi(w, b; x, y) = \max(0, 1 - y(w^T x + b))$ L1-loss
- $\xi(w, b; x, y) = \max(0, 1 - y(w^T x + b))^2$ L2-loss

in this project we will use the L2-loss version.
The variable $b \in \mathbb{R}$ is the bias term, which shifts the decision boundary away from the origin. The learned classifier, with w weights, assigns label $\hat{y} = \mathbf{sign}(w^T x + b)$ to a new instance x.

2.2 Handling the Bias Term

The bias term introduces an asymmetry between w and b in the primal objective and a linear equality constraint in the dual. A standard approach to avoid this complication is to embed b directly into the weight vector by appending a constant feature to each instance:

$$w \leftarrow \begin{bmatrix} w \\ b \end{bmatrix}, \quad x_i \leftarrow \begin{bmatrix} x_i \\ 1 \end{bmatrix} \tag{1}$$

After this operation, the bias is no longer a separate variable and the primal problem reduces to:

$$\min_w \frac{1}{2} w^T w + C \sum_{i=1}^{l} \xi(w; x_i, y_i) \tag{2}$$

where now $\xi(w; x, y) = \max(0, 1 - y w^T x)^2$, and the decision function is simply $\hat{y} = \mathbf{sign}(w^T x)$
This formulation, referred as the *unbiased* formulation, is adopted throughout this work. Its advantage is that the resulting dual problem is bound constrained, which allows the use of simple and efficient one variable coordinate descent update, as we will discuss in the next sections.
For completeness, we note that retaining b explicitly yields a dual with a linear equality constraint $\sum_i \alpha_i y_i = 0$, which forces any coordinate descent method to update at least two variable simultaneously. Chiu [2] analyze this case and show that the resulting CD method is significantly less efficient than its unbiased form, therefore we do not consider it further.

2.3 Dual Problem

Starting from the unbiased primal problem, the dual optimization problem is:

$$\begin{aligned} \min_\alpha f(\alpha) &= \frac{1}{2} \alpha^T \, \overline{Q} \alpha - e^T \alpha \\ \text{s.t.} \quad & 0 \le \alpha_i \le U, \ \forall i \end{aligned} \tag{3}$$

where $e = [1, ..., 1]^T$, $\overline{Q} = Q + D$, and $Q_{ij} = y_i y_j x_i^T x_j$. The matrix D and the upper bound U depend on the chosen loss function, for L2-loss:

$$U = \infty, \quad D_{ii} = \frac{1}{2C} \tag{4}$$

The diagonal term D_{ii} regularizes the problem and ensures that $\overline{Q}$ is positive definite even when Q is only positive semi-definite, but removes the upper bound on α_i. D is also needed to incorporate the Hinge-loss in the dual version. With an L1-loss we would have $D = 0$ and $U = C$.
The primal and dual solutions are connected through the relationship:

$$w = \sum_{i=1}^{l} \alpha_i y_i x_i \tag{5}$$

which holds at optimality. This identity is central to the efficient implementation of coordinate descent for linear SVMs: rather than storing the matrix $\overline{Q}$ explicitly, we can update and maintain the vector w throughout the optimization and compute the gradient of f as:

$$\nabla_i f(\alpha) = y_i w^T x_i - 1 + D_{ii} \alpha_i \tag{6}$$

This will take the computational cost from $O(l\bar{n})$ to $O(\bar{n})$ where $\bar{n}$ is the average number of nonzero features. This property is what makes coordinate descent methods competitive for large scale linear SVMs.

3 One-Variable Dual Coordinate Descent

3.1 Algorithm Description

The one variable DCD method, proposed by Hsieh et al. [1], solves the dual problem by cycling through all variables $[\alpha_1, ..., \alpha_l]$ and updating them one at a time, via $\alpha_i \leftarrow \alpha_i + d$, keeping the others fixed. Each update consists of solving a one dimensional (1D) subproblem of the form:

$$\begin{aligned} \min_d \frac{1}{2} \overline{Q}_{ii} d^2 + \nabla_i f(\alpha) \cdot d \\ \text{s.t.} \quad 0 \le \alpha_i + d \le U \end{aligned} \tag{7}$$

This is a simple quadratic in d with a box type constraint, and its solution is available in closed form:

$$\alpha_i \leftarrow \min\left(\max(\alpha_i - \frac{\nabla_i f(\alpha)}{\overline{Q}_{ii}}, 0), U \right) \tag{8}$$

where $\overline{Q}_{ii} = x_i^T x_i + D_{ii}$ can be precomputed once before the training loop.
At the start of each outer iteration, the variables are randomly permuted, so that the subproblems are solved in order $[\alpha_{\pi(1)}, ..., \alpha_{\pi(l)}]$.

3.2 Efficient Gradient Computation

The main computational challenge, with this kind of approach, is evaluating $\nabla_i f(\alpha)$ at each step. A naive computation would require accessing the *i-th* row of $\overline{Q}$, which costs $O(l\bar{n})$ per update and makes the method impractical for large datasets.

For linear SVMs, this cost can be reduced to $O(\bar{n})$ per update by maintaining the weight vector

$$w = \sum_{j=1}^{l} \alpha_j y_j x_j$$

throughout the optimization. The gradient then becomes:

$$\begin{aligned} \nabla_i f(\alpha) = (\overline{Q}\alpha)_i - 1 &= ((Q + D_{ii})\alpha)_i - 1 \\ &= ((y_i y_j x_i^T x_j + D_{ii})\alpha)_i - 1 \end{aligned} \tag{9}$$

`replacing the previous formulation of w`

$$= y_i w^T x_i - 1 + D_{ii}\alpha_i$$

$$\nabla_i f(\alpha) = y_i w^T x_i - 1 + D_{ii}\alpha_i$$

which requires only a dot product between w and x_i. With this trick we will never use Q explicitly. After updating α_i by step $\delta = \alpha_i^{new} - \alpha_i^{old}$, the weight vector is maintained incrementally:

$$w \leftarrow w + \delta \cdot y_i x_i$$

at a cost of $O(\bar{n})$. The total cost per outer iteration is therefore $O(l\bar{n})$, which is the same as reading all the data once.

3.3 Stopping Criterion

A variable α_i is already optimal for its subproblem if and only if its projected gradient $\nabla_i^P f(\alpha)$ is zero, where:

$$\nabla_i^P f(\alpha) = \begin{cases} \min(0, \nabla_i f(\alpha)) & \text{if } \alpha_i = 0 \\ \nabla_i f(\alpha) & \text{if } 0 < \alpha_i < U \\ \max(0, \nabla_i f(\alpha)) & \text{if } \alpha_i = U \end{cases} \tag{10}$$

At the end of each outer iteration k, we track two quantities:

$$M_k = \max_i \nabla_i^P f(\alpha^k), \quad m_k = \min_i \nabla_i^P f(\alpha^k) \tag{11}$$

By Theorem 2 of Hsieh et al. [1], $M_k - m_k \to 0$ as $k \to \infty$, and this quantity reaching zero is equivalent to global optimality. The stopping condition used in practice is therefore:

$$M_k - m_k < \epsilon$$

for a chosen tolerance $\epsilon > 0$.

4 Two-Variable Coordinate Descent

4.1 Motivation

The one variable DCD method updates a single dual variable at each step, leaving all others fixed. While this produces cheap updates, it may require many iterations to achieve convergence, particularly when the dual variables are strongly correlated, a situation that easily occurs for large values of C.

The two variable CD method address this limitation by updating pairs of variables (α_i, α_j) simultaneously at each step. Updating two variables at once allows the algorithm to move in directions that are inaccessible to single variable updates, potentially making much larger progress per step.

4.2 Pair Selection

At each outer iteration, the l training instances are randomly partitioned into $l/2$ consecutive pairs using a random permutation π. The pairs are $(\pi(1), \pi(2)), (\pi(3), \pi(4)), ..., (\pi(l-1), \pi(l))$. For each pair (i, j), a two dimensional subproblem is solved. This scheme of permutation ensures that every variable is updated exactly once per iteration.

4.3 Two Dimensional Subproblem

Given the current values α_i and α_j, and writing $d_i = \bar{\alpha_i} - \alpha_i$ and $d_j = \bar{\alpha_j} - \alpha_j$ for the two updates, the two variable subproblem is:

$$\begin{aligned} \min_{d_i, d_j} \frac{1}{2} \begin{bmatrix} d_i \\ d_j \end{bmatrix}^T \begin{bmatrix} Q_{ii} & Q_{ij} \\ Q_{ij} & Q_{jj} \end{bmatrix} \begin{bmatrix} d_i \\ d_j \end{bmatrix} + \begin{bmatrix} G_i \\ G_j \end{bmatrix}^T \begin{bmatrix} d_i \\ d_j \end{bmatrix} \\ \text{s.t.} \quad L_i \le d_i \le U_i\,, \quad L_j \le d_j \le U_j \end{aligned} \tag{12}$$

where the gradients and the matrix entries are:

$$G_i = y_i w^T x_i - 1 + D_{ii}\alpha_i, \quad G_j = y_j w^T x_j - 1 + D_{jj}\alpha_j$$

$$Q_{ii} = x_i^T x_i + D_{ii}, \quad Q_{jj} = x_j^T x_j + D_{jj}, \quad Q_{ij} = y_i y_j x_i^T x_j$$

and the bounds are $L_i = -\alpha_i, U_i = U - \alpha_i$ (analogously for j). The feasible region is a rectangle (or box constraint).

This is a quadratic problem with a positive definite Hessian and box constraints. While several approaches exist to solve it, the key of Chiu et al. [2] is that an exact solution can always be found efficiently by checking at most one optimality condition, without ever comparing objective values on different boundary segments.

4.4 Solution of the Two Dimensional Subproblem

The Hessian of the subproblem is:

$$H = \begin{bmatrix} Q_{ii} & Q_{ij} \\ Q_{ij} & Q_{jj} \end{bmatrix} \tag{13}$$

which is positive definite since $Q_{ii}, Q_{jj} \succ 0$ and the determinant of the Hessian is:

$$\delta = Q_{ii}Q_{jj} - Q_{ij}^2 > 0$$

The solution procedure follows Algorithm II of Chiu et al. [2] and proceeds in three main steps.

1. **Unconstrained minimum**: ignoring the box constraints, the unconstrained minimizer $d^* = (d_i^*, d_j^*)$ satisfies the linear system $Hd^* = -G$, which has a closed form solution:

$$d_i^* = \frac{-Q_{jj}G_i + Q_{ij}G_j}{\delta}, \quad d_j^* = \frac{-Q_{ii}G_j + Q_{ij}G_i}{\delta} \tag{14}$$

   The corresponding unconstrained optimal values are

$$\alpha_i^{free} = \alpha_i + d_i^*, \quad \alpha_j^{free} = \alpha_j + d_j^*$$

2. **Check feasibility**: if d^* lies in the box, it is the global minimizer and the procedure terminates. Otherwise, the solution lies on one of the boundary segments of the box, and we proceed to Step 3.

3. **Boundary solution via optimality conditions**: the projected point $\overline{d} = P[d^*]$ obtained by clipping each component of d^* to its feasible range is computed. The key results of Chiu et al. [2], established in Theorems II.1, II.2 and II.3, is the following:
   at least one of the two projected components $\overline{d}_i$ or $\overline{d}_j$ satisfies its own optimality condition.
   Specifically, at least one of the following holds:

$$Q_{ii}\overline{d}_i + Q_{ij}\overline{d}_j + G_i \begin{cases} \geq 0 & \text{if } \overline{d}_i = L_i \\ \leq 0 & \text{if } \overline{d}_i = U_i \end{cases} \tag{15}$$

$$\text{or} \quad Q_{ij}\overline{d}_i + Q_{jj}\overline{d}_j + G_j \begin{cases} \geq 0 & \text{if } \overline{d}_j = L_j \\ \leq 0 & \text{if } \overline{d}_j = U_j \end{cases}$$

The algorithm checks the optimality condition for d_i first. If it holds, d_i is fixed at $\overline{d}_i$ and d_j is obtained by solving the resulting one dimensional subproblem in closed form:

$$\overline{d}_j = \text{clip}\Big(- \frac{Q_{ij}\overline{d}_i + G_j}{Q_{jj}}, L_j, U_j \Big) \tag{16}$$

If the condition for d_i does not hold, then by Theorem II.3 it necessarily holds for d_j, so d_j is fixed at $\overline{d}_j$ and d_i is solved analogously:

$$\overline{d}_i = \text{clip}\Big(- \frac{Q_{ij}\overline{d}_j + G_i}{Q_{ii}}, L_i, U_i \Big) \tag{17}$$

In both cases we update the weight vector via:

$$w \leftarrow w + \overline{d_i} \cdot y_i x_i + \overline{d_j} \cdot y_j x_j$$

The interesting part of this approach is that it always finds the exact solution with at most one sign check, without evaluating the objective on multiple boundary candidates. A naive approach would instead enumerate all four boundaries and compare objective values (doing this for both indexes would be equal to checking 8 different cases).

4.5 Stopping Criterion

The stopping criterion for the two variable method mirrors that of the one variable method in Section 3.3. After each outer iteration, the projected gradients $\nabla_i^P f(\alpha)$ are recomputed for all i using the updated w, and the quantities M_k and m_k are evaluated as in Section 3.3. Convergence is reached when $M_k - m_k < \epsilon$.

4.6 Comparison with the One Variable Method

The two variable method requires computing $Q_{ij} = y_i y_j x_i^T x_j$ at each step, which is an additional dot product not present in the one variable method. As a result, each CD step costs approximately three dot product evaluations against one for the one variable method. Whether this extra cost is amortized by the reduction in the number of steps depends on the dataset. For datasets with many features and sparse representations, the dot product $x_i^T x_j$ is cheap and the two variable method tends to converge faster in runtime. For dense, low dimensional datasets, the additional cost is relatively more significant and the runtime advantage of the two variable method is less pronounced, consistent with the findings of Chiu et al. [2].

5 Implementation Details

5.1 Programming Languages

Both algorithms have been implemented in Python using NumPy, while the reference implementation provided by Chiu et al. [2] is written in C++. This difference has a significant impact on the observed performance. In particular, low level optimizations, memory management, and cache efficiency in C++ allow for substantially faster execution compared to a high level Python implementation. As a consequence, the run-time results obtained in this project are not directly comparable, in absolute terms, with those reported in the original paper. For this reason, the comparison between the one variable DCD and the two variable CD methods is carried out only within the same implementation framework, focusing on relative performance rather than the absolute run-time.

5.2 Evaluation of an Optimal Reference Value f^*

To plot the relative objective gap $\frac{|f(\alpha)-f^*|}{f^*}$, a reference optimal value f^* is needed. Since the exact optimum is not available analytically, f^* is approximated by running the one variable DCD and the two variable CD algorithm with a very tight tolerance (e.g. $\epsilon = 10^{-4}$) and a large number of iterations, until the stopping criterion is satisfied, then we pick the f value that is the minimum between the two executions. This approach is consistent with the procedure described in Chiu et al. [2], where α^* is defined as an

approximate optimal solution obtained by running many iterations of the algorithm.

5.3 Bias Term Embedding

As discussed in Section 2.2, the bias term is incorporated by appending a constant feature equal to 1 to each training instance. This is done inside the `fit()` method of both classes, so that the input data X passed by the user is never modified. A consequence of this choice is that sklearn's `LinearSVC` must be called with `fit_intercept = False` when used as a reference, since the bias is already encoded in the augmented feature vector.

5.4 Correctness Verification

The correctness of both implementations is verified by comparing the test accuracy obtained by the DCD and 2-CD algorithms against that of scikit-learn's `LinearSVC`, configured with the same loss function, penalty parameter C, and `fit_intercept = False`. Since both methods solve the same optimization problem, their test accuracies are expected to be within a fraction of a percent of each other at convergence. Significant discrepancies would indicate either a bug in the implementation or insufficient convergence.

6 Experiments

6.1 Datasets

The experiments are conducted on the **a9a** dataset, a binary classification benchmark derived from the UCI Adult dataset and available from the LIBSVM repository. The dataset contains 32,561 training instances and 16,281 test instances, each described by 123 binary and categorical features. The classification task is to predict whether an individual's annual income exceeds $50,000.

The experiments are also conducted on the **w8a** dataset, a binary classification benchmark available from the LIBSVM repository. The dataset contains 49,749 training instances and 14,951 test instances, each described by 300 numerical features. Unlike a9a, w8a is a synthetically generated dataset originally introduced by Platt (1999) to benchmark the Sequential Minimal Optimization (SMO) algorithm, and its features do not carry any semantic meaning.

Finally, we evaluated both algorithms on the **ijcnn1** dataset, a binary classification benchmark derived from the `Generalization Ability Challenge` of the IJCNN 2001 Neural Network Competition. The task consists in predicting the output of a synthetic time series, and has been widely used as a standard benchmark for SVM solvers. This dataset is also used in the experimental evaluation of Chiu et al. [2], making it a good choice for comparison. The dataset contains 49,990 training instances and 91,701 test instances, each described by 22 continuous features. Table 1 summarizes the datasets statistics.

Datasets	Train	Test	Features	NNZ/inst
a9a	32,561	16,281	123	13.8
w8a	49,749	14,951	300	11.6
ijcnn1	49,990	91,701	22	11.6

Table 1: Dataset statistics.

6.2 Experimental Setup

Both algorithms are run with penalty parameter $C \in \{1, 8192\}$ to evaluate behavior across different regularization regimes. The stopping tolerance is set to $\epsilon = 10^{-1}$ for the comparison runs. The reference value f^* is computed as described in Section 5.2. All experiments are run on a single MacBook Air M1 CPU core; no parallelism is exploited.

6.3 Accuracy Scores

As discussed in Section 5.4, we verify that both implementations produce models of comparable quality to scikit-learn's `LinearSVC`. Table 2 reports the test accuracy of the three methods on three different datasets for the two values of C.

Dataset	Method	$C = 1$	$C = 8192$
a9a	DCD	84.94%	77.53%
	2-CD	**84.96**%	78.48%
	LinearSVC	84.88%	**78.97**%
w8a	DCD	**98.67**%	**98.23**%
	2-CD	**98.67**%	98.17%
	LinearSVC	98.34%	97.32%
ijcnn1	DCD	91.78%	91.16%
	2-CD	**91.79**%	**91.98**%
	LinearSVC	**91.79**%	90.89%

Table 2: Test accuracy (%) on a9a, w8a and ijcnn1 for $C \in \{1, 8192\}$.

6.4 Convergence Analysis

6.4.1 Dataset a9a

Figures 1 report the convergence behavior of the two algorithms on a9a for $C \in \{1, 8192\}$. Each figure contains two plots: the left plot shows the relative gap $\frac{|f(\alpha)-f^*|}{f^*}$ as a function of the number of CD steps, while the right plot shows the same quantity as a function of runtime in seconds. The y-axis is on a logarithmic scale in both cases.

Case $C = 1$ (Figure 1, top). For this value of C the problem is relatively well conditioned and both methods

converge within epoch 110 to 135. In terms of CD steps (left plot), the two variable CD achieves a slightly lower objective gap than DCD for the same number of steps, confirming that each two variable update makes more progress. However, the improvement is modest: the two curves are close throughout, suggesting that for easy problem instances the additional expressiveness of two variable updates does not translate into a large reduction in the number of steps needed. In terms of runtime (right plot), DCD is faster than 2-CD. This is due to the higher cost per step of the two variable method: each 2-CD step requires three dot product evaluations compared to one for DCD, and on a9a (only 123 dense features) this overhead is not amortized by the modest reduction in steps. The use of Python further amplifies this effect, since the per step overhead of calling NumPy routines multiple times per iteration is non negligible compared to a compiled C++ implementation.

Case $C = 8192$ (Figure 1, bottom). For this larger value of C the problem is significantly harder and neither algorithm reaches the stopping criterion $M_k - m_k < \epsilon$ within the iteration budget. Notably, the same behavior is observed with scikit-learn's `LinearSVC` even with a large number of maximum iterations, confirming that the difficulty is intrinsic to the problem rather than a limitation of our implementation. The reason lies in the structure of the problem for large C: the diagonal regularization term $D_{ii} = \frac{1}{2C}$ becomes very small, making the Hessian $\overline{Q}$ nearly singular and the objective landscape very flat in certain directions. As a consequence, the projected gradients decrease slowly and the stopping condition is hard to satisfy. Furthermore, with large C almost all training instances become support vectors, meaning that a large number of dual variables are simultaneously active and each coordinate update makes only marginal progress.

Despite not converging, the results for $C = 8192$ are the most informative for comparing the two methods. In terms of CD steps (left plot), the 2-CD method achieves a substantially lower objective gap than DCD for the same number of steps, with the gap between the two curves being much larger than in the $C = 1$ case. In terms of runtime (right plot), 2-CD is also faster than DCD, a reversal of the pattern observed for $C = 1$. This is because for difficult problem instances the reduction in the number of steps provided by two variable updates is large enough to outweigh the higher per step cost, even in Python. This result is consistent with the findings of Chiu et al. [2], who report that the advantage of the two variable method is most visible for large values of C.

6.4.2 Dataset w8a

Figure 2 reports the convergence behavior of the two algorithms on w8a for $C \in \{1, 8192\}$.

Case $C = 1$ (Figure 2, top). For this value of C the problem is well conditioned and both methods converge, though requiring more iterations than on a9a: convergence is reached around epoch 324. In terms of CD steps (left plot), the two curves are nearly overlapping throughout, suggesting that on this dataset the two variable updates do not provide a meaningful reduction in the number of steps needed compared to one variable updates. Unlike a9a, where the curves follow a straight descending line in log scale, here both curves exhibit a nonlinear descending shape, reflecting the more complex geometry of the optimization landscape induced by the larger number of features and training instances. In terms of runtime (right plot), DCD remains faster than 2-CD, consistent with the observation on a9a: the additional cost of computing Q_{ij} at each step is not compensated by the modest reduction in steps for this value of C.

Case $C = 8192$ (Figure 2, bottom). As observed on a9a, neither algorithm reaches the stopping criterion $M_k - m_k < \epsilon$ within the iteration budget for this value of C, and the same behavior is confirmed by scikit-learn's `LinearSVC`. Despite not converging, both methods achieve substantially higher accuracy than on a9a, which can be attributed to the larger number of training instances and features available in w8a, allowing the model to learn a more expressive decision boundary. In terms of both CD steps (left plot) and runtime (right plot), the 2-CD method achieves a lower objective gap than DCD, confirming the pattern observed on a9a. Notably, the advantage of 2-CD appears more pronounced on w8a than on a9a: the 2-CD curve descends considerably faster than the DCD curve in both plots, suggesting that the larger problem size amplifies the benefit of two variable updates, as each step makes proportionally more progress relative to the one variable method.

6.4.3 Dataset ijcnn1

Figure 3 reports the convergence behavior of the two algorithms on ijcnn1 for $C \in \{1, 8192\}$.

Case $C = 1$ (Figure 3, top). The results closely mirror those observed on a9a: both methods converge around epoch 30 and the two curves behave similarly in terms of both CD steps and runtime, with DCD being slightly faster in run time due to the lower per-step cost.

Case $C = 8192$ (Figure 3, bottom). Neither algorithm converges within the iteration budget, consistently with the behavior observed on a9a and w8a. In terms of runtime (right plot), the advantage of 2-CD over DCD is minimal, likely due to the low dimensionality of the dataset: with only 22 features, the additional cost of computing Q_{ij} at each step is proportionally more significant and the problem is not complex enough to fully amortize it. In terms of CD steps (left plot), however, 2-CD achieves a lower objective gap than DCD for the same number of steps, confirming that two variable updates make more progress per step regardless of the dataset dimensionality.

Overall. Across all three datasets and both values of C, the results consistently show that the two-variable method is superior in terms of CD steps, while its runtime advantage depends on the difficulty and dimensionality of the problem. For large C, the benefit of 2-CD in both steps

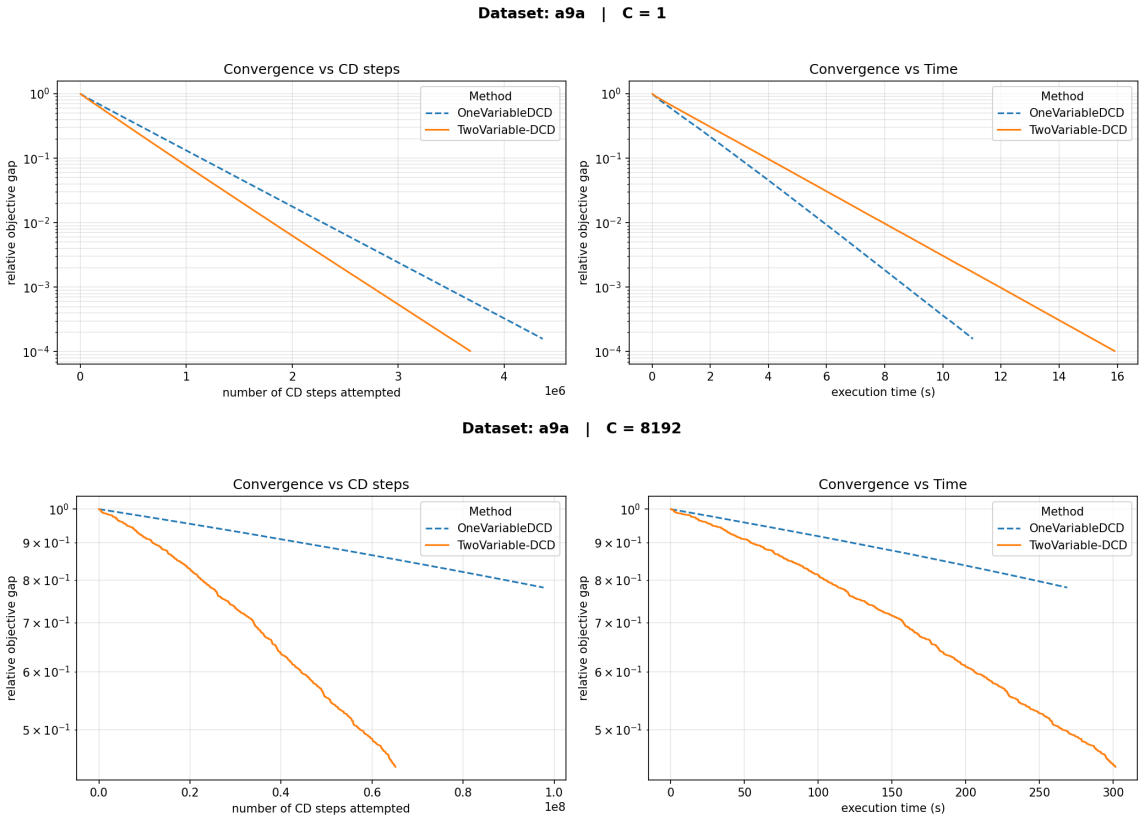

Figure 1: Relative objective gap vs. CD steps (left) and runtime (right) on a9a. Top: $C = 1$. Bottom: $C = 8192$.

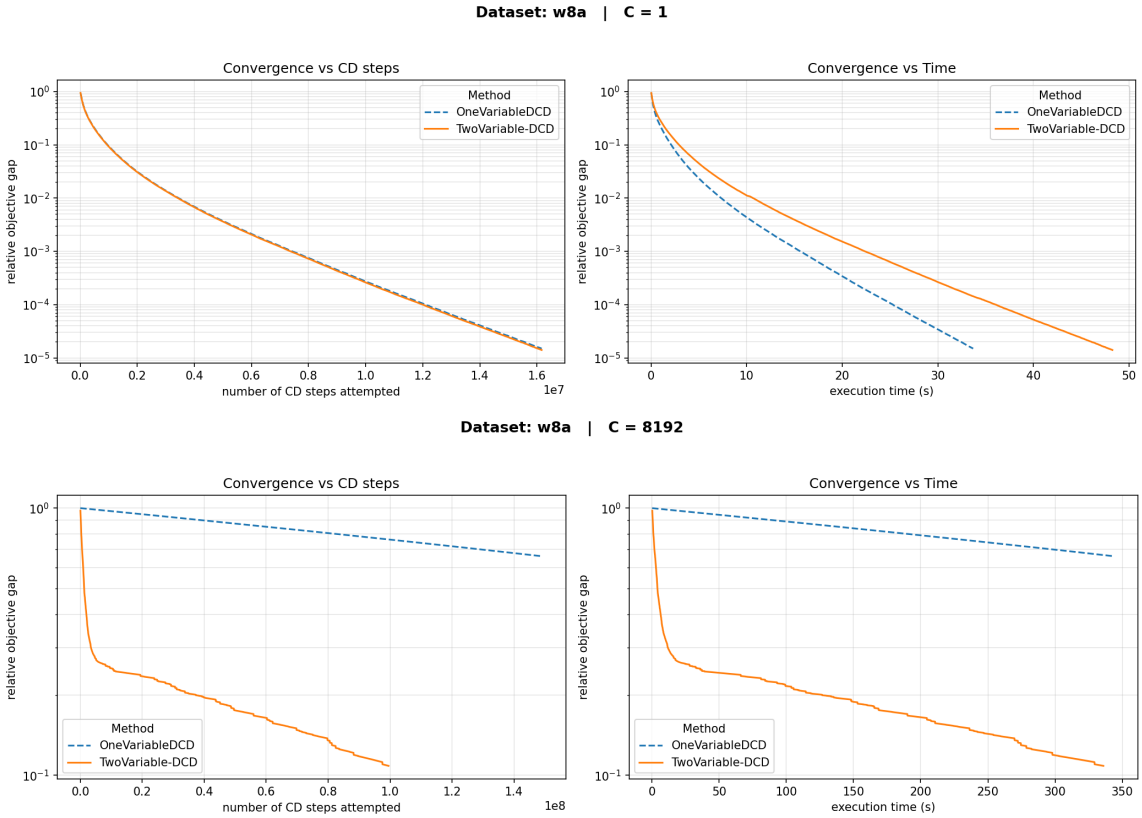

Figure 2: Relative objective gap vs. CD steps (left) and runtime (right) on w8a. Top: $C = 1$. Bottom: $C = 8192$.

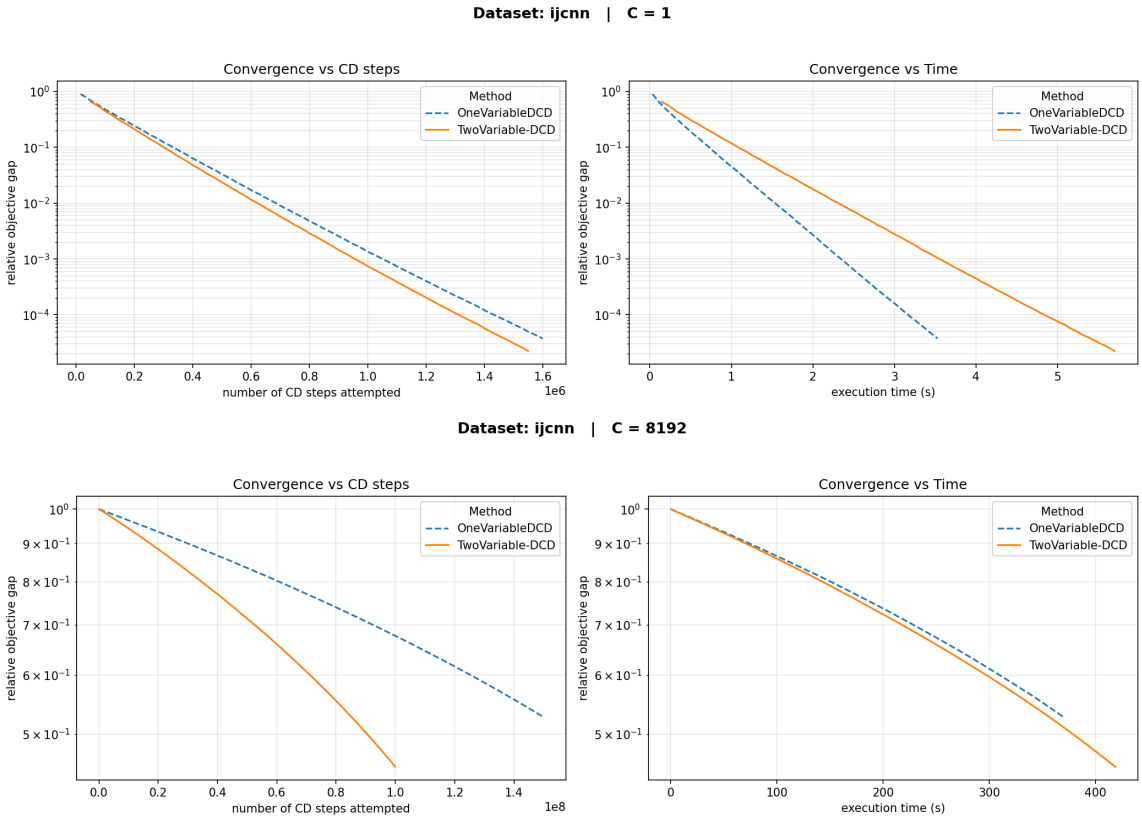

Figure 3: Relative objective gap vs. CD steps (left) and runtime (right) on ijcnn1. Top: $C = 1$. Bottom: $C = 8192$.

and runtime grows with problem complexity, as evidenced by the more pronounced gap observed on w8a compared to a9a and ijcnn1.

7 Conclusions

This project investigated and implemented two coordinate descent algorithms for training linear SVM via dual optimization: the one variable DCD method of Hsieh et al. [1] and the two variable CD method of Chiu et al. [2]. Both algorithms were implemented in Python using NumPy.
The experimental analysis revealed a clear dependence of the relative performance of the two methods on the difficulty of the problem, as controlled by the penalty parameter C. For small values of C, where the problem is well-conditioned, the one-variable DCD is competitive or superior in terms of runtime, due to the lower per-step cost. For large values of C, where the problem is harder and coordinate descent requires many steps to make progress, the two-variable 2-CD proves more efficient both in terms of CD steps and runtime, as the reduction in the number of steps outweighs the higher per-step overhead.

References

[1] Kai-Wei Hsieh, Cho-Jui Chang, and Chih-Jen Lin. Coordinate descent method for large-scale l2-loss linear support vector machines. *Journal of Machine Learning Research*, 9:1369–1398, 07 2008.

[2] Chi-Cheng Chiu, Pin-Yen Lin, and Chih-Jen Lin. *Two-variable Dual Coordinate Descent Methods for Linear SVM with/without the Bias Term*, pages 163–171. 01 2020.